ASTRA 2.0: An Evidence-Guided Campaign Controller for Multi-Model Scientific Research

Architecture and engineering report

16 August 2026

Abstract

Large language models can produce scientific prose far faster than they can produce scientific *evidence*. ASTRA is a research engine that closes that gap by forcing every claim through an executable falsification step: a heterogeneous ensemble proposes, one model authors a validator, a second and independent model reviews it, and a deterministic oracle runs it. ASTRA 2.0 adds a campaign layer above that atomic cycle—an append-only evidence ledger, structured proposal portfolios, hard admissibility gates, and progressive widening—so that a research programme, not just a single question, can be driven to a decision.

This report describes the architecture, the epistemic commitments that shape it, and the algorithm that implements them. It then documents, in detail, the failures encountered while making the system measurable and then while running it. That second part carries most of the engineering content. Of the defects found during evaluation, most were in the *measurement infrastructure* rather than in the reasoning engine, and three of four benchmark runs measured the harness instead of the system: we report a cross-agent instruction injection present in 72% of deliberations, a phase-budget allocation that starved the one step that converts a rejected validator into a result, and an output-token ceiling consumed entirely by internal reasoning. Four further defects appeared only once the campaign layer drove real research, because a benchmark counts outcomes while a campaign acts on them—including an episode that recorded supporting evidence for a claim from a cycle that had answered an unrelated question. Five more surfaced from a single sustained campaign driven to one recorded verdict, and—arriving in sequence on one unbroken problem—they let us test the report's central claim directly: that keeping the status axes separate distinguishes a failure of transport from a failure of science. In one episode a one-part-in-10^{11} transcription error in a consensus conjecture was stopped from being recorded as a refuted scientific claim by a single axis declining to collapse into another. We also report, without softening it, a comparative evaluation that remains inconclusive and whose first measured signal runs against our own hypothesis.

Scientific results obtained with the system are deliberately not reported here; where live research exposed a structural defect, the defect is described and the science is left to its own project.

1 Introduction

1.1 The problem

An ensemble of strong language models will reliably generate a plausible research narrative for almost any prompt. It will far less reliably generate a *decidable* statement, a validator that could refute it, and an honest verdict when the refutation succeeds. The failure modes are well known: the model answers a question adjacent to the one asked; numerical sampling is presented as proof of a universal claim; a crash is reported as a scientific refutation; a corrected version of a false claim is validated and reported as if the original had been confirmed.

ASTRA is built on the premise that these failures are architectural, not prompting problems, and that the cure is to make refutation *cheap, mandatory and mechanical*. Every cycle must end with something a machine ran.

1.2 Scope: commercially available models

ASTRA does research with the models anyone can buy—subscription-tier CLIs from three providers, on one workstation, under ordinary quota. It is not built on, and makes no claim about, the private or unreleased higher-capability systems the provider companies hold internally.

That is the premise of the design rather than a limitation to apologise for, and every distinctive choice descends from it. Independence is bought by *heterogeneity* instead of scale: three model families with different blind spots give a cross-check no single model can give itself. Refutation is *mechanical* because no available model reliably reports when its own argument fails, so a deterministic auditor, an executable validator and an oracle stand where self-assessment would otherwise go. Budget discipline is *structural*—phase ceilings, downstream reserves, a machine-wide single-cycle lock—because quota is scarce and shared rather than elastic.

The consequence deserves stating plainly: a substantially stronger model would not make this architecture unnecessary, only cheaper to run, because fewer repair rounds would be needed. The question it answers is *where evidence comes from*, and that question does not dissolve with model capability.

1.3 What ASTRA already does

The production system implements one *atomic cycle*: a bounded investigation of a single decidable proposition. Two heterogeneous models propose independently, critique each other, and a synthesiser merges them into one consensus conjecture. A third model—deliberately not one of the proposers—writes an executable validator. A deterministic auditor and then an independent model reviewer attack that validator before it runs. Only then does an oracle execute it, and only then does an analyst adjudicate the result.

ASTRA 2.0 does not replace that worker. It changes how proposals, branches, evidence and subsequent cycles are *represented and selected*, so the system can pursue an objective across many cycles without losing the minority proposals, the refuted branches, or the reasons for either.

1.4 Contributions of this report

1. An architecture in which epistemic status is deliberately *not* collapsed into a single number (Section 3).
2. A campaign algorithm combining hard admissibility gates with a visible, unlearned priority vector (Section 4).
3. A catalogue of measured failure modes in multi-agent research systems, including a cross-agent instruction injection that is a structural consequence of a correct safety choice (Section 5).
4. An honest evaluation, including a negative-leaning first signal and the reasons it cannot yet be trusted (Section 6).

2 The atomic cycle

2.1 Topology

Figure 1 shows the cycle. Three properties of the layout are load-bearing rather than incidental.

The author is not a proposer. The model that writes the validator did not propose the conjecture. A model asked to test its own idea will write a test its idea passes.

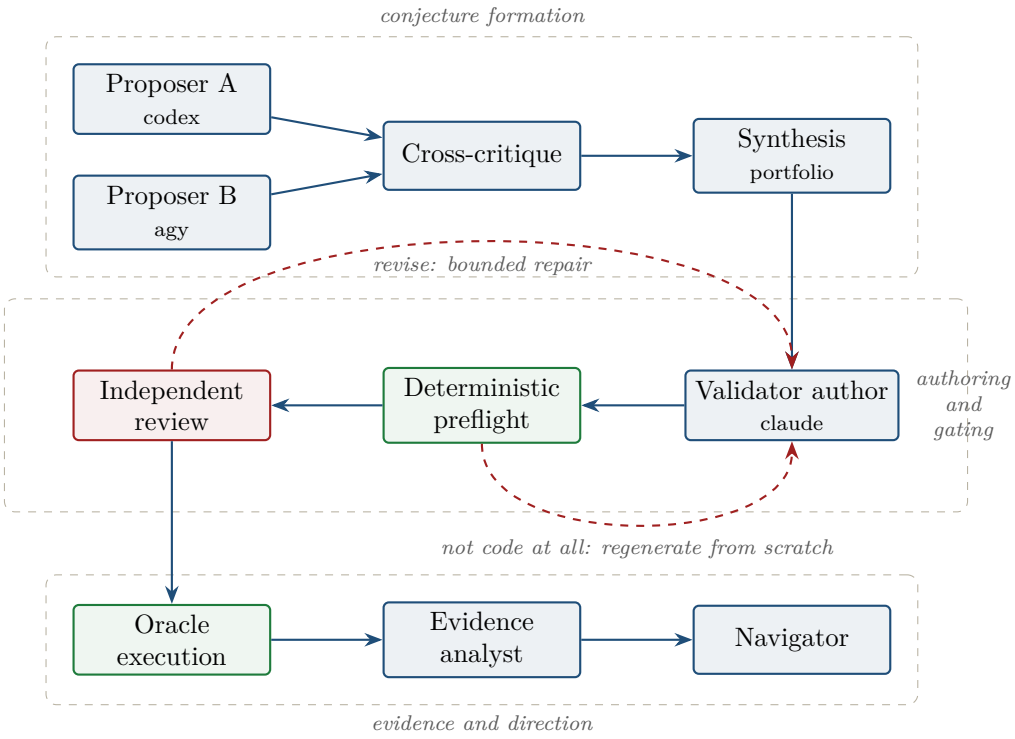

Figure 1: The atomic cycle. Blue: model phases. Green: deterministic steps. Red: gates that can send work back. The dashed red arrows are the two return paths that Section 5.5 shows must be kept distinct.

A deterministic auditor runs before the model reviewer. Parsing, static analysis and a smoke run cost milliseconds and catch the defects that are cheapest to catch. The expensive reviewer only sees code that already compiles and can, in principle, fail.

The reviewer is independent of the author. The reviewer belongs to a different provider and model family, so it does not share the author's blind spots by construction.

2.2 The self-refutation harness

The validator is not asked to confirm the conjecture. It is required to test it through independent legs and to print one line per leg:

- a *symbolic* leg: an exact residual or identity reduced to a literal zero;
- a *numeric* leg: evaluation at seeded random points against a tight tolerance;
- a *limit* leg: a degenerate case with a known closed answer.

The script prints `VERDICT: PASS` only if every leg passes. Critically, the failing branch must be *reachable code*: a deterministic AST auditor rejects validators that cannot fail, and the cycle is re-run against the author with the auditor's reasons. A test that cannot fail is not evidence.

3 Epistemic architecture

3.1 Five axes that must not collapse

The single most consequential design decision in ASTRA is that status is not one value. Five distinct questions are tracked separately (Figure 2):

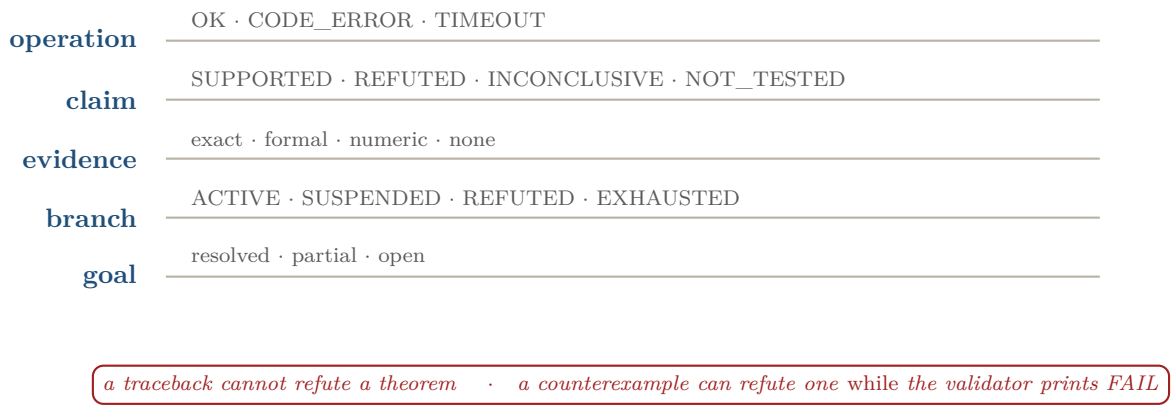

Figure 2: The five status axes are recorded independently for every episode. The box states the two facts that make the separation necessary in both directions.

1. **operation status** — did the phase or tool complete?
2. **claim status** — supported, refuted, inconclusive, not tested?
3. **evidence grade** — what kind and strength of evidence exists?
4. **branch status** — should this direction continue?
5. **goal coverage** — how much of the objective is resolved?

Collapsing any pair produces a specific, observable pathology:

Collapse	Pathology
operation into claim	a Python traceback "refutes" a theorem
numeric into evidence grade	sampling silently becomes a universal proof
claim into goal coverage	one lemma reports the programme as finished
claim into branch	a refuted branch closes a still-viable direction

3.2 Verdict re-anchoring

An early live measurement exposed a subtle version of the same disease. Given a false claim, ASTRA would correctly identify the falsehood, form a *corrected* conjecture, validate the correction, and report `VALIDATED`. Every individual step was right; the reported status answered the conjecture the system had formed rather than the claim the user had made.

The fix is a separate axis, `original_claim_verdict`, taking values SUPPORTED, REFUTED, INCONCLUSIVE, SUBSTITUTED or UNSPECIFIED, produced by the analyst from the user's original statement. Verified live on seeded false claims, it moved the false-acceptance rate from an apparent 0.67 to a true 0.0: the earlier number was an instrument artefact, not a hallucination rate.

3.3 Evidence policy by claim type

Different claim shapes admit different minimum evidence. This is enforced, not advisory.

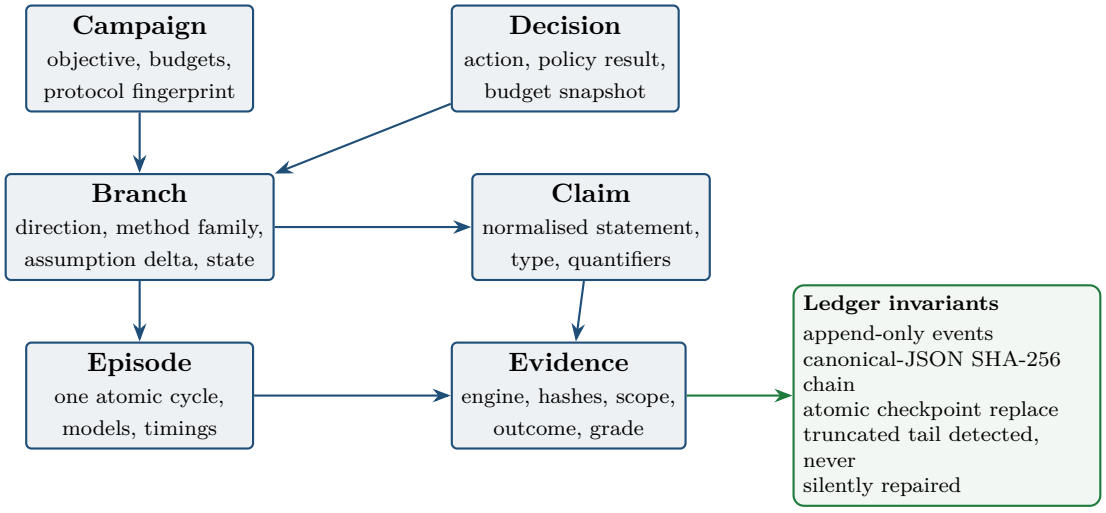

Figure 3: The six live records and the ledger guarantees behind them. A partially written tail is detected and archived rather than repaired, because silently repairing an audit log destroys the property that makes it an audit log.

Claim type	Minimum credible evidence	Escalation
Universal theorem	exact argument, or explicitly bounded scope	CAS/SMT, then a proof kernel
Existential / refutation	one exact admissible witness	minimality, independent reconstruction
Equivalence / derivation	conditions plus exact residual	independent CAS, high precision
Numerical prediction	reproducibility, uncertainty, convergence	independent implementation
Algorithmic guarantee	proof obligations plus sanity cases	formalisation of decisive lemmas
Scoped empirical claim	frozen data and protocol, stated uncertainty	replication, holdout

An existential claim needs one witness; a universal claim needs an argument. The asymmetry is not decoration: in live use it is the difference between an obligation a cycle can discharge and one it cannot, and a campaign that states its claims in the wrong shape spends its budget on validators the reviewer will correctly refuse (Section 5.7).

4 The campaign layer

4.1 Data model

Six append-only record types (Figure 3) are enough to drive a programme. Everything else—method, provenance, review, artifact data—is a typed field or a linked manifest. The graph is a derived projection; the event log is the source of truth.

4.2 Portfolios instead of a single winner

Ensemble synthesis normally destroys information: the minority proposal disappears into the consensus text. ASTRA 2.0 requires the synthesiser to return a *structured portfolio*—the selected atomic claim plus two to four materially different admissible alternatives, each with its method family, its assumption delta, and the cheapest evidence that would discriminate it from the selected one.

The parser is deliberately asymmetric: *fail-soft* on the portfolio as a whole (a malformed block must never destroy a paid-for cycle) and *fail-closed* on individual candidates (a candidate without a crux is not admissible). One live defect came from exactly this boundary: a model emitted an empty string inside a quantifier list and an early implementation discarded the entire portfolio.

4.3 Hard gates, then a visible priority vector

A scalar reward is epistemically unsafe: scientific truth, novelty, stability and cost are not one number, and a numerical failure may mean a false claim, an unstable representation, or simply a broken validator. ASTRA 2.0 therefore applies *hard admissibility gates* first, and only then orders the survivors by a visible, unlearned priority vector.

Hard gates (all must hold)	**Priority vector (lexicographic)**
relevant to an unresolved deliverable	expected information gain
falsifiable or formally scoped	current evidence strength
explicit domain, assumptions, quantifiers	methodological independence
feasible evidence plan	goal advancement
no exhausted duplicate method or claim	estimated execution and model cost
sufficient remaining budget	instability and hallucination risk

The Navigator model supplies estimates and rationale for the vector, but **cannot override a failed gate**. A model recommendation that contradicts a deterministic gate is recorded and ignored—this is enforced in code and pinned by a regression test.

4.4 Progressive widening

Only one branch is active initially. Another is promoted only on a specific trigger: negative or inconclusive evidence, method redundancy, evidence saturation, or a high-value independent alternative. Breadth must be *earned by evidence*, because a controller that widens on enthusiasm produces many branches and no results.

4.5 The algorithm

Campaign step

1. Freeze brief, budgets, resources and protocol hash.
2. Run the parallel proposal and cross-critique phase.
3. Require a structured portfolio from synthesis; promote minority candidates to PROPOSED branches.
4. Execute *only* the selected claim through authoring, preflight, review, oracle, audit and navigation.
5. Classify the outcome: scientific refutation, invalid validator, operational error, numerical instability, formalisation failure, or credible evidence.
6. Append episode and evidence manifests to the ledger.
7. Let the models recommend; enforce the deterministic constraints. *Never* promote a branch with a missing quantifier contract; *never* treat sampling as proof; *never* treat an operational failure as a refutation; *never* declare the goal resolved while mandatory deliverables remain.
8. Continue the active branch while evidence is informative and the next obligation is clear; widen only when a trigger fires.
9. Trigger the expanded adversarial campaign only at a scientific milestone.
10. Terminate on complete result, structured partial progress, exhausted search, budget exhaustion, or an explicit human decision.

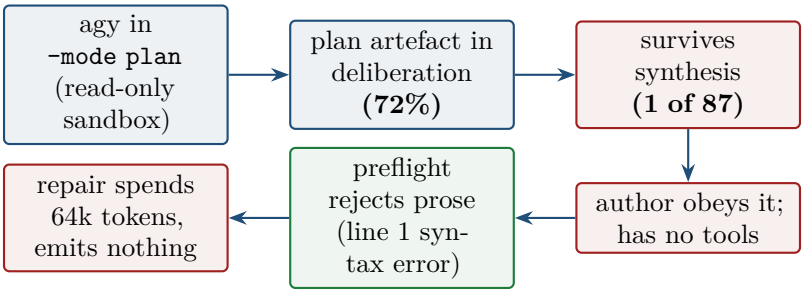

Figure 4: The causal chain from a deliberate safety setting to a destroyed cycle. Text produced by one agent became instructions for another; the benign origin does not change the mechanism.

5 Failure modes we had to solve

This section is the empirical core of the report. The defects below were found by running the system, not by reading it, and each one is accompanied by the measurement that exposed it.

5.1 Cross-agent instruction injection

Observation. One deliberation ended with the following sentence carried into the consensus conjecture:

```
Review the proposed implementation plan artifact [plan.md](file:///...)
      to formally construct the validator.  Pending your approval.
```

The validator author runs with all tools disabled, by design. It nevertheless attempted to comply, emitting roughly forty repetitions of "`**Tool call:** let me read it`" instead of a script. That prose entered the pipeline as code, failed to parse, and the repair call that followed consumed its entire 64 000-token output budget without emitting a single character (Figure 4).

Diagnosis. The leak is not a model quirk. ASTRA invokes the Antigravity CLI with `-mode plan` *deliberately*: it is the read-only sandbox that prevents that CLI from writing files or executing, exactly as `-tools ""` does for the author. Plan-mode artefacts are therefore the *structural cost of a correct safety choice*. Measured over 87 deposited checkpoints, the artefact appears in **72% of deliberations**; synthesis filtered it out in every case but one, and that one destroyed the cycle.

Fix. Two layers. The conjecture now travels to the author inside an explicit fence, and a prompt rule states that anything inside the fence demanding a tool call is ensemble scaffolding to be ignored. Independently, the artefact is stripped at the source, matched on the invariant (a `file://` link) rather than on the wording, which varies and changes language. Removal is always announced; a reply that is *only* an artefact pointer is reported as a failed call rather than cleaned, because the model left its answer in a file the system cannot read.

5.2 Budget allocation: a share cannot protect the end of a pipeline

Observation. Every phase call was capped at a fraction of the *remaining* cycle budget, so that no phase could starve its successors. Over a canary run, all twelve validator repairs timed out, at a median ceiling of 165 s against the 480 s configured, and always with budget still on the clock. The split between success and failure was purely temporal: cycles that validated ran a median of 882 s, cycles that failed 1249 s.

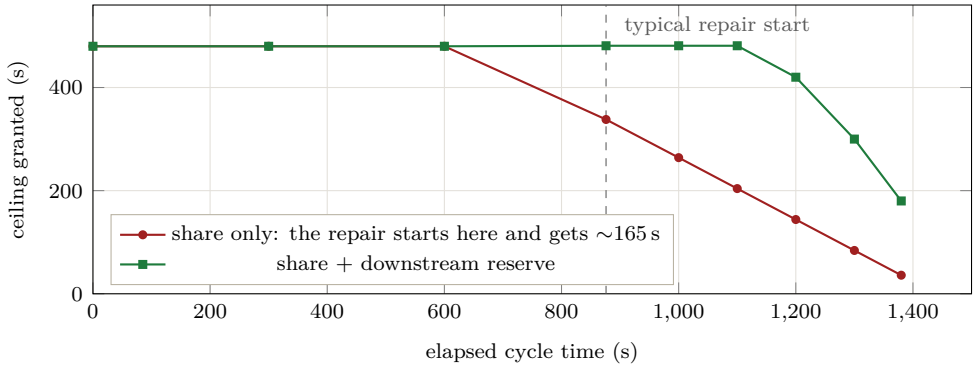

Figure 5: Ceiling granted to the validator repair as a function of when it starts. Under a pure share rule the last phase is systematically the poorest; the reserve restores it. Points are computed from the measured phase medians.

Diagnosis. A share is a fraction of what remains, so it shrinks as the cycle progresses and hands the least to the phase that runs last. The validator repair *is* last, and it is the step that converts a rejected validator into a result. A proportional rule starves precisely the phase whose starvation is most expensive (Figure 5).

Fix. Phases additionally declare what they must *leave behind*. The reserve is sized from measured p90s: 343 s for the repair plus 270 s for execution, analysis and navigation. After the change, repairs received their full 481 s ceiling in every observed case.

5.3 The pipeline has a loop, and phase-indexed state assumes a line

Three separate defects shared one root cause: state indexed by *phase name* used to reason about *position in the flow*. The validation pipeline is not linear—review runs again after every repair—so any rule of the form "phase X must reserve for what follows X" is wrong on the second pass.

The clearest instance: a final review round was refused with 571.83 s still available, because it was asked to hold back 613 s for a repair that could no longer run, the revision budget already being spent. The reserve now answers "what can still run?" rather than "which phase is asking?".

A companion guard was added at the same time. A phase whose computed ceiling falls below 45 s is not started at all: issuing a call that cannot succeed burns quota and, worse, mislabels budget exhaustion as a model timeout. Before the guard, the logs contained `timeout tras 1s`, which reads as a hung model and is nothing of the sort.

5.4 An output ceiling consumed by reasoning

A repair call was killed after 1200 s having produced 184 KB of internal reasoning and *zero* characters of answer, stopping at exactly 64 000 output tokens. Raising the time ceiling from 480 s to 1200 s had changed nothing, because the binding constraint was never time.

Sizing the fix required care in both directions: the calls that *worked* had spent 59 181 and 56 630 tokens, so a cap below that would degrade the one configuration known to function. The thinking budget is capped at 58 000, which clears every observed success and still reserves about twice the largest validator ever measured.

5.5 Prose is not a damaged script

A reply that fails to parse can be a broken script or not a script at all, and the two need opposite treatments. Handing prose back as "your previous code, fix line 1" is an impossible instruction—it was the request that consumed the 64 000-token budget above. The preflight now distinguishes

the cases and is deliberately conservative: any `import` or `def` counts as an attempted script, so a validator with a typo keeps its right to a local repair, and only genuine prose triggers regeneration from a clean slate.

5.6 What only live research exposed

The defects above were found by benchmarking. Four more surfaced only when the campaign layer drove an actual research programme, and none of them could have appeared in a benchmark, because a benchmark *counts* outcomes while a campaign *acts* on them.

A gate that declines is not a tool that broke. The independent reviewer refused a validator—correctly, and with specific objections—and the campaign recorded it as `operation_status: FAILED`. Two such episodes then promoted a second branch under progressive widening: a scientific decision taken from an operational signal, which is the collapse of Figure 2 happening inside the code written to prevent it. A refusal now maps to a completed pipeline with an untested claim; a reviewer that times out stays an operational failure. Same phase, opposite meaning.

Widening must not be driven by episodes that never ran. The same rule counted an episode that produced no phases, no models and no timings—the signature of a lost lock. A tool that broke says nothing about the promise of a research direction. The rule now skips such episodes without letting them extend or reset a streak, and deliberately still counts a validator that ran and crashed, because that *does* say the direction is hard to test.

A replayed cycle is not an observation. Two consecutive episodes recorded byte-identical timings and two separate evidence records: the second step re-sent the same direction, hit the cycle cache and returned in seconds. The cache is right for a research loop revisiting a direction and wrong for a ledger, where it manufactures corroboration. Replays are now refused before they can reach the record.

An episode is evidence only if the cycle ran for that claim. The executor accepts a cycle result through a runner hook, which is useful for tests and for recovering a cycle whose runner died. A result from an entirely unrelated project—running in the same minute, on the same workstation—was fed in through that hook, and the ledger gained a SUPPORTS record whose validator tested a completely different mathematical object. Every status axis, every gate and every hash was correct about a result that answered another question. Results are now bound to the request by their shared objective, and the ledger records the retraction rather than hiding it.

Defect	Invisible to benchmarks because	Structural fix
reviewer refusal recorded as breakage	benchmarks count it, campaigns act on it	five axes enforced at the campaign boundary
widening driven by a lost lock	benchmarks have no branches to widen	episodes that did no work are skipped
cached replay minted as evidence	benchmarks disable the cache	replays refused before the ledger
foreign result accepted by the runner hook	benchmarks never hand-feed results	result bound to the request's objective

5.7 Claim shape decides what a cycle can discharge

A campaign spent three episodes without a single validator admitted, and the reviewer's three refusals were each correct. The cause was not the models: every branch obligation had been stated as an existential over a *continuum*. Such a claim is not decided by evaluation—satisfying it needs

a witness, refuting it needs an argument over the whole family—so the author kept attempting the middle route of sweeping and summarising, and the reviewer kept rejecting it.

Restating each obligation at a concrete point produced a decided result in one episode. A second restatement mattered as much: asking whether a parameter *moves* the quantity, rather than whether it solves the problem, so that the claim cannot be settled without the parameter being involved. The first formulation had produced a refutation whose witness was numerically identical to the baseline—a correct verdict about a sentence, establishing nothing about the idea.

This is a protocol property, not a physics one. The evidence policy of Section 3 already said an existential is discharged by a witness; what was missing was any mechanism that stopped a campaign from seeding claims its own policy could not serve.

5.8 Failure attribution is infrastructure

Over 87 deposited checkpoints, **7 cycles** reported a failure of the *reviewer* when the component that actually died was the validator author, and in all seven the generated validator was overwritten by the error string. The consequence is not cosmetic: it corrupts the very statistics used to tune the system. Under the old labels the picture read "the reviewer fails 15 of 15"; under corrected labels it read "the repair starves in 8 of 15"—and only the second statement points at a fixable cause.

Symptom	Apparent cause	Actual cause
reviewer fails 15/15	the reviewer	repair starved by a share rule
`timeout tras 1s`	a hung model	budget exhausted; call issued anyway
translator needs more time	a small ceiling	output tokens spent on reasoning
cycle dies writing a validator	model incapacity	injected instruction from a peer agent
both arms fail 100% of cycles	the architectures	a stale lock; BUSY scored as failure

5.9 A sustained campaign, and the thesis it tested

The defects of Sections 5.6–5.8 surfaced in the first hours of campaign use. A longer test followed: one research question driven across two campaigns and eleven episodes, to a single recorded verdict. It exposed five further defects, and—because they arrived in sequence on one unbroken problem—it did more than lengthen the list. It tested this report's central claim, that keeping the five axes separate (Figure 2) lets the system tell a failure of *transport* from a failure of *science*. Each of the five is infrastructure misread as a research signal, and in each the separation is what contained it.

A provider outage counted as an uninformative episode. The streak rule of Section 5.6, which pauses a branch after two episodes that teach nothing, used the absence of timings as its proxy for "did no work." A provider returned HTTP 500 and killed the author after 935 s of conjecture and translation, so the episode carried timings and the outage counted—tripping the two-episode pause one episode early, on the strength of one genuine refusal plus one server error. The discriminator is not "did the episode spend time" but "did it record evidence about the direction," a line the axes already draw: an API error yields no evidence outcome, a code error an operational-error record, a refusal an inconclusive one. A failed episode that recorded no evidence is now skipped; one that recorded evidence, or completed, counts.

A branch could not read what its ancestor had certified. The request handed to a cycle carried the campaign objective, the branch direction and the branch's first claim—and nothing else. A branch created by progressive widening on an earlier result therefore could not

receive that result. One branch published an exact matrix and reached SUPPORTED; its successor, whose obligation began "given that matrix," was handed no matrix, and its author re-derived the assembly the split existed to avoid—reproducing the exact failure the split had removed. The request now carries, for the active branch, the verbatim certified output of every SUPPORTED episode in its ancestry, under an explicit instruction to load rather than re-derive it.

A malformed repair patch destroyed an approved validator. A validator passed the independent reviewer and executed with `VERDICT: PASS`; the analyst then returned `CODE_ERROR` while its own written reasoning affirmed the proof, having read a Sage deprecation warning on `stderr` as a code fault. That spurious error triggered a repair, the repair model returned a reply that was not valid JSON, and the cycle took its failure path—erasing an approved, passing validator that had certified the result. Two changes close it. The deterministic guard, which already downgraded an over-confident VALIDATED, now reconciles the reverse: a `CODE_ERROR` raised over a clean `VERDICT: PASS` with no correction to offer is the analyst mislabelling a warning, and is restored. And a failed or inapplicable patch now keeps the pre-patch validator and finalises on the honest status, rather than destroying a cycle that a malformed reply should never have been able to destroy.

A reviewer's objection did not survive to the next author. The symmetric partner of the ancestor-context defect above. Over one branch the reviewer refused four validators, each for a different and correct reason: re-deriving the data, gating on an exact value, deciding a rational sign outside `QQbar`, reducing to a representative tensor. Each objection lived only in the cycle checkpoint, unseen by the next fresh author, who invented the next shortcut; one episode reproduced two earlier shortcuts at once. The obligation was never wrong—the reviewer's feedback converged, repeatedly calling the mathematics sound—but that feedback evaporated between episodes. A withheld reviewer's reasoning is now persisted into the evidence record and surfaced to the next episode on the same branch, so a retry answers the objection instead of discovering a new way to fail.

A Sage success that called `sys.exit` was misread as an error. The last barrier was the smallest. A validator satisfied every clause of a demanding contract—literal data, every sign in `QQbar`, the verdict resting only on the decisive leg, the exact certificate printed—and the reviewer approved it. It ended its passing run with `sys.exit(0)`, and SageMath's runner reports a raised `SystemExit` as a nonzero exit, so the printed `VERDICT: PASS` was recorded as an operational error. The contract now tells every author to complete naturally on the success path; operational failures still exit nonzero, the distinction being success versus failure rather than exit versus return.

The separation, caught in the act. One episode shows the thesis working rather than being argued for. The ensemble's consensus conjecture stated the minimum of a quadratic form as 40864160967/722297701575; the true value, computed from the data, is 40864160968/722297701575—larger by one part in 7×10^{11}, and both strictly positive. A validator gated on the stated value, found the mismatch, and returned a refutation. Had `claim_status` been the only axis, the ledger would now read *the null-energy condition is refuted.* It does not: the re-anchoring axis recorded `original_claim_verdict: SUBSTITUTED`, marking the refutation as one of a mis-transcribed constant—a substituted question—while the physical claim, the sign of the minimum, was untouched. Figure 2 is not decoration; here a one-part-in-10^{11} transcription slip was stopped from masquerading as a scientific result by a single axis refusing to collapse into another.

The quantity under test was confirmed by six independent routes before the clean verdict was ever recorded; what took two campaigns and eleven episodes was not the science but a single verdict that met every requirement at once, blocked at each step by one of the five artefacts above rather than by the physics. Each was fixed at its source, and each now serves any campaign rather than

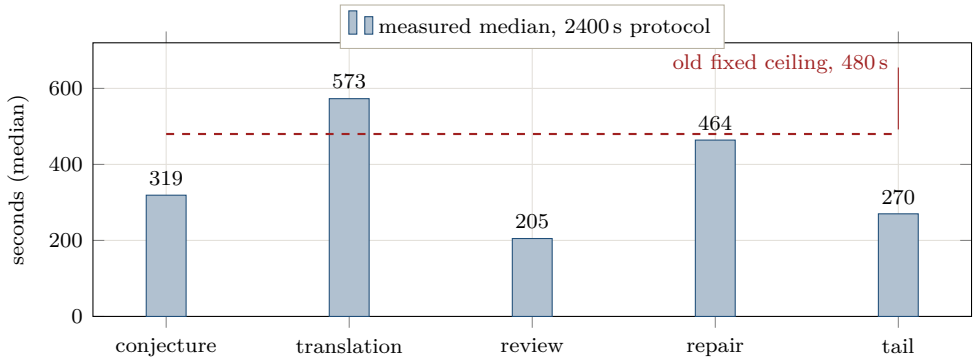

Figure 6: Phase medians over 18 cycles. The dashed line is the ceiling that was in force: below the median of the phase it was supposed to bound.

the one that exposed it. The scientific result itself belongs to its own project and is not reported here.

Defect	**Misread as**	**Structural fix**
provider 500 with timings	an uninformative research episode	streak keys on recorded evidence, not elapsed time
ancestor result not in the request	a direction that must re-derive	certified ancestor output carried to the successor
malformed patch on an approved validator	a cycle to be failed	guard reconciled both ways; last good validator kept
reviewer objection lost between episodes	a fresh, uninformed retry	objection persisted and surfaced to the next author
Sage `sys.exit(0)` on success	an operational error	contract: complete naturally on the passing path

6 Evaluation

6.1 What the measurement cost

Four benchmark runs were required to obtain one valid dataset. The first three measured the harness: a stale lock (a transient `BUSY` scored as an architectural failure, producing an apparently meaningful failure = 1.0 in *both* arms within 45 seconds); a scheduler context that broke the sandbox mechanism of one CLI, failing every process launch; and an expired CLI session. Each produced a number with the shape of a finding and the content of an artefact.

This is the report's most transferable lesson. In a multi-agent system whose components are subscription CLIs on a developer workstation, *the environment is an experimental variable.* A forty-second pre-flight that verifies every provider answers before committing hours of quota would have saved three runs.

6.2 Phase cost

Figure 6 shows measured phase medians on the amended protocol. Two facts follow. The pipeline is dominated by conjecture formation and validator authoring, jointly about 58% of a cycle; and a fixed ceiling sized against an older budget stops meaning anything when the budget moves—the translator ceiling of 480 s sat *below* the 573 s median of the task it bounded, and four of eight timeouts died at exactly that value. Ceilings that must track the cycle are now expressed as a fraction of it.

6.3 The comparative result, unvarnished

The preregistered question is whether the heterogeneous reflective loop (`full`) produces better research trajectories than a matched linear control (`full-linear`) under equal budgets. We fixed in advance that the comparison would only be interpretable below an operational failure rate of 0.2.

	cycles	validated	breakage	reviewer rejections
`full`	8	**0**	5 (0.63)	2
`full-linear`	10	**3**	4 (0.40)	3

The criterion is not met, so the comparison does not license a conclusion. We report the numbers anyway, including the part that runs against our own hypothesis: with 63% and 40% breakage, which arm validates depends mostly on where the breakage fell, and `full` had fewer chances because it died earlier (16 model calls per cell against 26). With four cells per arm, 0 against 3 is compatible with chance. It is nevertheless the first measured signal, and it does not point the way we assumed.

Note also the distinction the aggregate hides: *breakage* (timeouts, exhausted budget) is a reliability defect, whereas a *reviewer rejection* is the system refusing to certify weak work. Counting the second as a failure penalises rigour. A short verification run after the fixes described above produced four cycles, four validations and zero breakage in both arms—which establishes that no known structural blocker remains, and, honestly, nothing more: no translation in it exceeded 296 s, so the new ceiling was never exercised.

7 Why we believe this helps research

Refutation is the default, not an option.
The validator must be able to fail, and a deterministic auditor rejects it when it cannot. This inverts the usual incentive of a generative system, which is to produce a satisfying answer.

Independence is structural.
The author is not a proposer, the reviewer is not the author, and the oracle is not a model at all. Agreement between them carries information precisely because it was not arranged.

Status cannot be flattened.
The five axes prevent the specific confusions that make automated research untrustworthy—most importantly, that a program crash is not a scientific result and that answering a corrected question is not answering the question asked.

Evidence is executable and deposited.
Every claim is backed by a script, its hash, its output and the model that wrote it. A reader can re-run the evidence rather than trust the narrative.

Negative results are first-class.
Pruning a hypothesis family, exhausting a method, or reporting INCONCLUSIVE are recorded outcomes with the same standing as a success. Most real research time is spent here, and systems that only recognise successes silently discard it.

8 Limitations

- **The central comparison is unresolved.** No valid pilot has yet met the reliability precondition; the only measured signal is small and adverse. We claim an architecture and a failure catalogue, not a demonstrated uplift.

- **Blinded expert scoring is absent.** The primary quality endpoint requires two human raters per cell, blind to architecture. Automatic metrics measure process and efficiency, not scientific depth, and we decline to substitute one for the other.
- **Small n.** Canary tiers use two cases and two seeds. Several conclusions drawn earlier in this project at n=1 reversed on repetition, and are marked as retracted in the deposited evidence.
- **Environment coupling.** The system depends on subscription CLIs whose authentication, sandboxing and process behaviour are part of the experimental apparatus, as Section 6 shows at some cost.
- **Single-workstation quota.** One deliberative cycle runs at a time, machine-wide, because concurrent cycles share model accounts. This bounds throughput independently of the architecture.

9 Reproducibility

Every number in this report is traceable to a deposited artefact: phase timings and status counts come from per-cycle JSON checkpoints; validator sources, oracle stdout and their hashes are stored beside them; the benchmark protocol is content-addressed, and an amendment to it (widening the per-cycle budget from 1800 s to 2400 s) changed that fingerprint, was dated, justified and recorded in a fingerprint history rather than applied silently. The regression suite stands at 435 tests, and the defects described in Section 5 are each pinned by a test that replays the measured values that exposed them.

Note on scope. This report describes an engineering system, its protocol and its evaluation. Scientific results obtained *with* ASTRA belong to their own projects and are not reported here; where live research exposed a structural defect, the defect is described and the physics is not. A companion volume of worked examples is planned separately.